ChatGPT

HashX Global V1 Product Requirements Document

Executive Summary

HashX Global is a verification-first agent-to-agent commerce layer built around HashKey Chain and HSP. Its core proposition is deliberately narrower than a generic crypto payment gateway: **an AI agent should not be able to pay merely because another agent asks for money; it should first be able to prove what was requested, who requested it, whether the invoice was altered, whether an objectively testable digital service was delivered, what the human ultimately authorized, and what HSP actually settled.**

The locked first use case is **an AI buyer purchasing a digital service from an AI seller**. The seller delivers first; HashX verifies the signed invoice and deterministic fulfillment evidence; a human approves the exact payment; HSP executes settlement; HashX reconciles the resulting HSP state and exposes a verifiable receipt. The first demo service should be a small **HashKey Chain analytics report**, because the output can be checked against chain data instead of asking an LLM to subjectively decide whether work is "good." HashKey Chain currently exposes an EVM-compatible mainnet at chain ID `177` and testnet at chain ID `133`, making standard Solidity, Foundry and EVM-signature tooling usable. [1]

The project should be entered under **Agent Payments & A2A Settlement**, with HSP as the actual payment rail rather than a decorative integration. The October 30 HSP QA guide requires a public egress IP allowlist, exactly eight HSP tools, live-schema inspection, manual wallet/token preparation, explicit human confirmation before signing and again before submission, preservation of `flow_id` and `payment_request_id`, and polling until a terminal payment state. filecite turn0file1

The strongest competitive positioning is:

> **AllScale asks whether a merchant should trust an agent. Sumplus asks whether an agent is allowed to spend. PayPort simplifies merchant checkout. Astrail demonstrates a vertical agent purchase. HashX asks whether the commercial obligation itself is authentic, fulfilled under explicit machine-verifiable terms, safely authorized, and correctly reconciled to settlement.**

That differentiation is credible only if the demo contains real negative tests and a real HSP payment. AllScale already won a previous Horizon DeFi track with a merchant-side agentic-commerce gateway, and its public implementation performs EIP-712 payment authorization and settlement verification; therefore HashX cannot win merely by presenting "AI agent + signed payment + dashboard." [2]

Product status

Area	Decision
Product	HashX Global V1

Area	Decision
Track	Agent Payments & A2A Settlement
Primary buyer	AI agent acting for a human/company
Primary seller	AI agent offering objectively verifiable digital work
Human role	Final financial approver
First service	HashKey Chain address/block-range analytics report
Exchange order	Delivery → deterministic verification → user authorization → HSP settlement
Invoice security	EIP-712 signed structured invoice
Settlement rail	HSP Agentic Payment QA
Public-chain evidence	Minimal commitment registry only
Custody	HashX never stores raw user private keys
Main innovation	Commercial-verification layer before settlement
Primary demo	Tampered invoice rejection + bad delivery rejection + duplicate/replay rejection + valid HSP payment
Submission deadline	**November 1, 2026, 21:30 IST**, based on organizer information supplied for this hackathon; organizer confirmation remains advisable where DoraHacks rules conflict

The most important external validation gates remain **live HSP connectivity/payment execution, exact HSP live schemas, and organizer interpretation of the mainnet-deployment requirement**. HSP explicitly instructs teams to inspect current tool schemas rather than hard-code assumptions, so the request structures below are HashX-owned API contracts; the HSP adapter must map them dynamically to the currently exposed HSP schema. filecite turn0file1

Product Definition, Personas, Journeys, and MVP

Product vision

Vision: become the trust layer through which autonomous software can transact with other autonomous software without reducing commerce to “recipient + amount + signature.”

V1 value proposition: HashX binds five pieces of evidence into one auditable commercial flow:

`Commercial Intent → Signed Invoice → Verified Delivery → Human Authorization → Reconciled Settlement`

The value is not that invoices are “on-chain.” The value is that **the same immutable commercial terms used to verify delivery are linked to the exact payment the human approves and to the payment outcome reported by HSP**.

A plain blockchain transaction proves money moved. An EIP-712 signature proves a signer approved structured data. Neither alone proves that a seller delivered an agreed service. EIP-712 explicitly provides typed structured-data signing and domain separation but does **not** itself provide replay protection, so replay prevention must be enforced by HashX. [3]

Prioritized personas

Priority	Persona	Primary goal	Key concern	HashX responsibility
P0	Buyer Agent	Procure digital work and prepare payment	Fraudulent invoice / bad delivery	Verify before payment eligibility
P0	Seller Agent	Deliver service and request settlement	Invoice integrity / proof of delivery	Issue signed commercial terms and delivery manifest
P0	Human Approver	Decide whether funds may move	Agent compromise / hidden recipient or amount	See exact terms and explicitly approve signing + submission
P1	Integrator	Add HashX commerce to an AI product	Stable schemas and recovery	MCP/API contracts, typed errors, status recovery
P2	Operations/ Admin	Investigate failed or disputed transactions	Missing evidence	Audit trail and settlement explorer

Buyer-agent journey

The buyer agent receives an objective work request such as “analyze address X over blocks A–B.” It requests a quote/invoice from the seller agent, calls `verify_invoice`, waits for the seller's delivery, invokes `verify_delivery`, and only if the deterministic verifier accepts the artifact does it call `request_payment`. HashX then creates or retrieves the HSP payment workflow. The human must separately approve the exact HSP signing payload and submission step because the current HSP hackathon flow requires both confirmations. 【filecite【turn0file1】

Seller-agent journey

The seller creates a structured invoice, signs it from its registered wallet, performs the service, and submits the deliverable plus a manifest binding the artifact to the invoice and service parameters. It does **not** receive payment automatically merely because it claims delivery.

Human-approver journey

The human sees a compact approval card containing merchant/team attribution, seller/payee, payer, token, raw authorization amount, human-readable amount, invoice value, delivery verification result, network, quote expiry and whether the amount differs from the original service price due to payment fees. HSP explicitly requires displaying the payee, payer and `token_total_raw` before signature and showing them again before submission. filecite turn0file1

Integrator journey

An integrator connects to HashX's MCP server over Streamable HTTP, authenticates, discovers the eight HashX tools, and uses strict JSON-schema inputs. MCP's official TypeScript SDK supports tools, resources, prompts and Streamable HTTP; current v2 is the stable line implementing the 2026-07-28 MCP specification. [4]

Locked P0 MVP

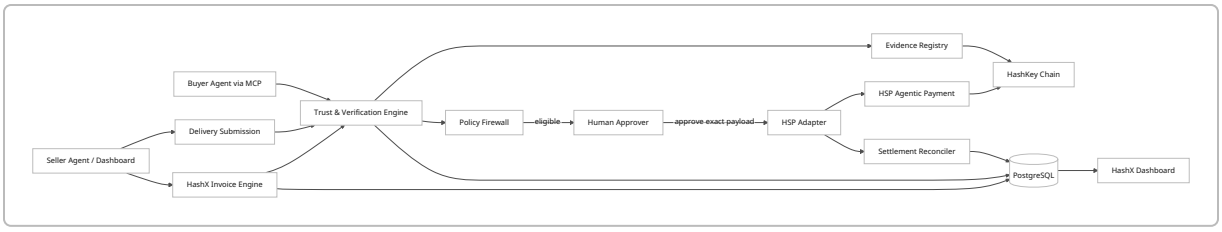
Must ship:

P0 capability	Acceptance condition
Manual invoice creation	User can create a valid unsigned invoice in dashboard
Agent invoice creation	MCP can create the same schema
Wallet invoice signature	EIP-712 signature generated outside HashX key storage
Invoice verification	Detects signer mismatch, mutation, expiry and replay
Trusted-counterparty binding	Registered seller wallet can be resolved to expected seller record
Seller delivery submission	JSON artifact + manifest attached to exact invoice
Deterministic fulfillment verifier	Valid analytics artifact passes; malformed or incorrect artifact fails
Payment-eligibility policy	No HSP order unless invoice and delivery pass
HSP adapter	Uses live HSP tools and current schemas
Two-stage human approval	Before HSP signing and before HSP submission
Idempotency	Repeated `request_payment` does not silently create another order
Payment recovery	Existing identifiers survive timeout/restart
Settlement reconciliation	`query_payment` state shown accurately
Verifiable receipt	Invoice + delivery evidence + HSP outcome linked
Evidence registry	Minimal public commitment written to HSK testnet/mainnet as appropriate

P0 capability	Acceptance condition
Dashboard	Invoice Workspace, Trust Inspector, Settlement Explorer
Adversarial demo	At least three attack/failure tests plus golden path
Repository quality	README, architecture, tests, deployments, limitations

Out of scope: fiat on/off ramps, global banking rails, arbitrary real-world-service adjudication, escrow, seller-guaranteed payment, refunds, chargebacks, tax accounting, full KYC network, universal reputation, stealth payments, zero-knowledge privacy, cross-chain routing, autonomous approval-free spending, proprietary wallet custody, subscriptions, multi-party invoice splitting, marketplace discovery, token issuance and trading.

HashKey Merchant already supports one-time and reusable payment orders, signed webhooks and multiple chains/tokens, so rebuilding generic checkout infrastructure is strategically weak for this hackathon. 5



Interfaces, EIP-712 Specification, and Data Model

HashX MCP/API design

The following are **HashX-owned contracts**. They should remain stable even if HSP changes its live MCP argument names.

Tool	Mutates state	Authorization	Idempotency key
`create_invoice`	Yes	Seller/integrator	`client_request_id`
`sign_invoice`	Yes	Seller wallet interaction	Invoice ID + version
`verify_invoice`	No*	Buyer/integrator	N/A
`submit_delivery`	Yes	Seller	Invoice ID + delivery version
`verify_delivery`	No*	Buyer/system	N/A
`request_payment`	Yes	Buyer	Invoice ID
`get_payment_status`	No	Buyer/seller	HSP IDs
`get_receipt`	No	Authorized participant	Invoice ID

`verify_*` may persist an audit event but must not alter the signed commercial object.

API schemas

`create_invoice`

```
{
  "client_request_id": "uuid",
  "seller_counterparty_id": "uuid",
  "buyer_ref": "string",
  "service": {
    "type": "hsk_address_analytics_v1",
    "parameters": {
      "address": "0x...",
      "start_block": "uint64-string",
      "end_block": "uint64-string"
    },
    "acceptance_policy": "analytics_v1"
  },
  "payment": {
    "currency": "USD",
    "amount_minor": "100",
    "payee": "0x...",
    "preferred_chain_id": 133
  },
  "expires_at": "RFC3339"
}
```

Response:

```
{
  "invoice_id": "uuid",
  "invoice_version": 1,
  "status": "DRAFT",
  "typed_data": {},
  "invoice_digest": "0x...",
  "created_at": "RFC3339"
}
```

`sign_invoice`

The tool should **prepare** a signing request and record a returned signature; it must not expose a private-key operation.

```
{
  "invoice_id": "uuid",
```

```
  "signature": "0x65-byte-signature",
  "signer": "0x..."
}
```

Response:

```
{
  "invoice_id": "uuid",
  "status": "SIGNED",
  "signature_valid": true,
  "signer": "0x...",
  "signed_digest": "0x..."
}
```

`verify_invoice`

```
{
  "invoice_id": "uuid",
  "expected_seller_id": "uuid",
  "expected_buyer_ref": "string"
}
```

Response:

```
{
  "decision": "PASS",
  "checks": {
    "schema": "PASS",
    "signature": "PASS",
    "identity_binding": "PASS",
    "expiry": "PASS",
    "nonce_unused": "PASS",
    "terms_consistent": "PASS"
  },
  "invoice_digest": "0x...",
  "reason_codes": []
}
```

`submit_delivery`

```
{
  "invoice_id": "uuid",
  "artifact_uri": "private-object-reference",
```

```
  "artifact_sha256": "hex",
  "manifest": {
    "schema": "hashx.delivery.analytics.v1",
    "seller": "0x...",
    "generated_at": "RFC3339",
    "service_parameters_hash": "0x...",
    "artifact_sha256": "hex"
  },
  "manifest_signature": "0x..."
}
```

`verify_delivery`

```
{
  "invoice_id": "uuid",
  "delivery_id": "uuid"
}
```

Response:

```
{
  "decision": "PASS",
  "verifier": "analytics_v1.0.0",
  "checks": {
    "manifest_signature": "PASS",
    "artifact_hash": "PASS",
    "schema": "PASS",
    "requested_address": "PASS",
    "block_range": "PASS",
    "transaction_count": "PASS",
    "sample_transaction_proofs": "PASS",
    "freshness": "PASS"
  },
  "evidence_digest": "0x...",
  "reason_codes": []
}
```

`request_payment`

```
{
  "invoice_id": "uuid",
  "payer": "0x...",
```

```
  "merchant_name": "HashX Global"
}
```

Successful pre-sign response:

```
{
  "payment_attempt_id": "uuid",
  "state": "AWAITING_SIGNATURE_APPROVAL",
  "flow_id": "...",
  "payment_request_id": "...",
  "pay_to": "0x...",
  "payer": "0x...",
  "token_total_raw": "...",
  "quote_expires_at": "RFC3339",
  "eip712": {},
  "approval_summary": {}
}
```

This operation MUST return the existing payment attempt for the invoice if a nonterminal attempt already exists.

`get_payment_status`

```
{
  "invoice_id": "uuid"
}
```

Response:

```
{
  "payment_attempt_id": "uuid",
  "hsp_status": "payment-processing",
  "flow_id": "...",
  "payment_request_id": "...",
  "attribution_verified": true,
  "tx_hash": null,
  "last_queried_at": "RFC3339"
}
```

`get_receipt`

```
{
  "invoice_id": "uuid"
}
```

Response:

```
{
  "receipt_version": "1",
  "invoice_digest": "0x...",
  "seller": "0x...",
  "buyer_ref": "...",
  "delivery_evidence_digest": "0x...",
  "payment": {
    "hsp_status": "payment-finalized",
    "flow_id": "...",
    "payment_request_id": "...",
    "tx_hash": "0x...",
    "payer": "0x...",
    "payee": "0x...",
    "token_total_raw": "..."
  },
  "evidence_registry_tx": "0x..."
}
```

EIP-712 invoice typed-data specification

EIP-712 supports structured signing with domain separation, and `chainId` should be incorporated when the signing domain is chain-specific. Because HashX invoices are commercial documents rather than HSP token authorizations, **HashX must use a distinct signing domain from HSP** to avoid semantic confusion. [3]

Recommended domain:

```
{
  "name": "HashX Global",
  "version": "1",
  "chainId": 133,
  "verifyingContract": "0x<HashXEvidenceRegistry>"
}
```

Recommended primary type:

```
HashXInvoice(
  bytes32 invoiceId,
  uint256 version,
  address seller,
  bytes32 buyerRefHash,
  bytes32 serviceType,
  bytes32 serviceTermsHash,
  uint256 amountMinor,
  bytes32 currency,
  address payee,
  uint256 paymentChainId,
  uint256 nonce,
  uint64 issuedAt,
  uint64 expiresAt
)
```

Typed-data JSON:

```
{
  "types": {
    "EIP712Domain": [
      {"name":"name","type":"string"},
      {"name":"version","type":"string"},
      {"name":"chainId","type":"uint256"},
      {"name":"verifyingContract","type":"address"}
    ],
    "HashXInvoice": [
      {"name":"invoiceId","type":"bytes32"},
      {"name":"version","type":"uint256"},
      {"name":"seller","type":"address"},
      {"name":"buyerRefHash","type":"bytes32"},
      {"name":"serviceType","type":"bytes32"},
      {"name":"serviceTermsHash","type":"bytes32"},
      {"name":"amountMinor","type":"uint256"},
      {"name":"currency","type":"bytes32"},
      {"name":"payee","type":"address"},
      {"name":"paymentChainId","type":"uint256"},
      {"name":"nonce","type":"uint256"},
      {"name":"issuedAt","type":"uint64"},
      {"name":"expiresAt","type":"uint64"}
    ]
  },
  "primaryType": "HashXInvoice",
  "domain": {},
```

```
  "message": {}
}
```

For V1, signer verification should support EOAs. ERC-1271 support is a P1 extension for smart-contract sellers; ERC-1271 defines `isValidSignature(bytes32,bytes)` and the `0x1626ba7e` success value for contract-based signatures. [6]

Data model and database proposal

PostgreSQL is appropriate because HashX needs transactional uniqueness guarantees around invoices and payments.

Table	Core fields	Important constraints
`counterparties`	`id`, `display_name`, `wallet_address`, `wallet_type`, `status`, `created_at`	unique normalized wallet
`wallet_proofs`	`id`, `counterparty_id`, `challenge`, `signature`, `verified_at`	challenge single-use
`invoices`	`id`, `version`, `seller_id`, `buyer_ref_hash`, `service_type`, `service_terms_json`, `amount_minor`, `currency`, `payee`, `chain_id`, `nonce`, `expires_at`, `digest`, `signature`, `status`	unique `(seller_id, nonce)`; unique digest
`deliveries`	`id`, `invoice_id`, `version`, `artifact_uri`, `artifact_sha256`, `manifest_json`, `manifest_signature`, `status`	unique `(invoice_id, version)`
`verification_runs`	`id`, `object_type`, `object_id`, `verifier_version`, `decision`, `checks_json`, `evidence_digest`, `created_at`	append-only
`payment_attempts`	`id`, `invoice_id`, `flow_id`, `payment_request_id`, `payer`, `payee`, `token_address`, `token_total_raw`, `quote_expires_at`, `hsp_status`, `tx_hash`	partial unique active attempt per invoice; unique HSP IDs
`approvals`	`id`, `payment_attempt_id`, `stage`, `actor`, `terms_hash`, `approved_at`	unique `(attempt,stage)`
`receipts`	`id`, `invoice_id`, `receipt_json`, `receipt_digest`, `registry_tx`	unique invoice

Table	Core fields	Important constraints
`audit_events`	`id`, `aggregate_type`, `aggregate_id`, `sequence`, `actor`, `event`, `payload_digest`, `created_at`	unique aggregate sequence
`idempotency_keys`	`key`, `scope`, `request_hash`, `response_ref`, `expires_at`	primary key `(scope,key)`

The most important database invariant is: **there may never be two simultaneously payable HSP attempts for one invoice**. Enforce this with a transaction plus a partial unique index, not a UI-level “already paid?” check.

Verification Engine, HSP Mapping, and Security Model

Invoice identity binding

HashX V1 should use **wallet-backed trusted-counterparty identity**, not claim universal legal identity verification.

A counterparty onboarding flow should:

1. create a random challenge containing HashX domain, wallet, nonce and expiration;
2. require wallet signature;
3. recover the EOA address;
4. bind that wallet to a HashX counterparty record;
5. require buyer-side selection of the expected seller record;
6. reject invoices whose recovered signer differs from the trusted seller wallet.

The distinction is important: a wallet proof establishes control of a wallet, not legal incorporation, KYC status or merchant honesty.

Signature verification

HashX recomputes the EIP-712 digest from canonical invoice fields and verifies the submitted signature against the seller wallet. Any change to amount, payee, service terms, buyer binding, expiration, nonce or version must invalidate the invoice. EIP-712's domain separation and structured data are appropriate here, but replay prevention must be implemented separately. [3]

Replay protection and duplicate prevention

Use four layers:

- unique seller nonce;
- immutable invoice version;
- invoice expiry;
- DB-level uniqueness for active payment attempts.

A settled invoice becomes `SETTLED`; a failed HSP attempt can be retried only according to a controlled state transition. An uncertain HSP response must be recovered via `query_payment` using the same identifiers rather than creating another order. This directly follows HSP's documented recovery rule. 🗄filecite🗄turn0file1🗄

Deterministic analytics-delivery verifier

Locked service: seller receives `{address,start_block,end_block}` and returns a JSON report.

Recommended artifact:

```
{
  "schema": "hashx.analytics.v1",
  "chain_id": 133,
  "address": "0x...",
  "start_block": 123,
  "end_block": 456,
  "transaction_count": 12,
  "native_value_in_raw": "...",
  "native_value_out_raw": "...",
  "counterparties": [
    {"address":"0x...","tx_count":4}
  ],
  "sample_transactions": [
    {"hash":"0x...","block_number":234}
  ],
  "generated_at": "RFC3339"
}
```

Acceptance policy:

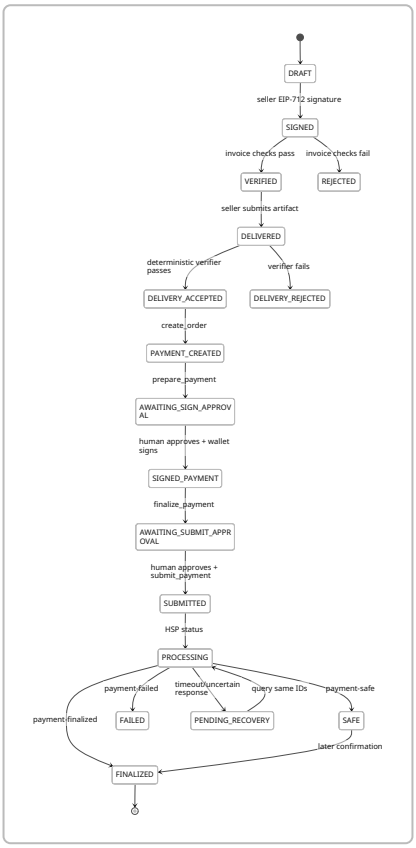
Check	Rule	Failure code
Schema	Exact supported version	`DELIVERY_SCHEMA_INVALID`
Chain	Equals invoice requested chain	`CHAIN_MISMATCH`
Address	Normalized exact match	`SUBJECT_MISMATCH`
Range	Exact requested start/end	`BLOCK_RANGE_MISMATCH`
Artifact hash	Hash equals signed manifest	`ARTIFACT_TAMPERED`
Seller manifest	Valid wallet signature	`MANIFEST_SIGNATURE_INVALID`
Tx count	Recompute against HSK RPC	`TX_COUNT_MISMATCH`

Check	Rule	Failure code
Sample txs	Each tx hash exists in declared range and involves address	`EVIDENCE_INVALID`
Freshness	Generated after requested end block is available	`DELIVERY_STALE`
Completeness	Required fields present	`DELIVERY_INCOMPLETE`

HashKey publishes RPC endpoints for testnet and mainnet, so the verifier can recompute selected analytics directly against chain RPC rather than trusting seller output. [7]

Important limitation: validating a bounded set of analytics facts does not prove arbitrary report quality. The demo and documentation must say exactly what HashX verified.

Transaction lifecycle



HSP integration mapping

HSP QA uses Streamable HTTP at the documented MCP endpoint and requires all eight tools: `get_chain_config`, `create_order`, `get_order`, `prepare_payment`, `finalize_payment`, `estimate_fee`, `submit_payment`, and `query_payment`. Teams must inspect their live schemas before invocation and must not invent missing fields. filecitefileturn0file1

HashX stage	HSP operation	HashX rule
Adapter startup	`get_chain_config`	Cache briefly; use returned network/token/protocol only
Eligible invoice	`create_order`	Bind `pay_to` to signed invoice; stable `merchant_name`
Order validation	`get_order`	Reconfirm order details before preparation
Optional fee preview	`estimate_fee`	Display, never substitute for locked quote
Payment preparation	`prepare_payment`	Persist response before wallet signing
Signature	External wallet	Sign exact returned `eip712`, nothing reconstructed
Finalization	`finalize_payment`	Pass returned objects unchanged + wallet signature
Submission	`submit_payment`	Only after second explicit approval
Recovery/finality	`query_payment`	Same IDs; never infer success

`get_chain_config` is authoritative for token address, decimals, chain and whether the protocol is `eip3009` or `permit2`; the payer must hold the exact token contract returned. filecite turn0file1

`create_order` requires the team `merchant_name`, separately confirmed `pay_to`, and a cart mandate using USD, x402 v2 and current configuration values. It returns `flow_id`, `payment_request_id` and `payment_url`; HashX must persist both IDs. filecite turn0file1

`prepare_payment` returns the exact EIP-712 object, `cart_hash`, `token_total_raw`, quote expiry and payment objects. HSP says the quote is valid for five minutes; expired quotes or Gateway code `30043` require a fresh `prepare_payment` and fresh wallet signature. filecite turn0file1

Payment status handling:

HSP state	HashX behavior
`payment-submitted`	Continue polling
`payment-verified`	Continue polling
`payment-processing`	Continue polling
`payment-included`	Continue polling
`payment-timeout`	Recover same payment; do not recreate
`payment-safe`	Show “on-chain / not finalized”
`payment-finalized`	Mark final

HSP state	HashX behavior
`payment-failed`	Stop and preserve reason
unknown	Show unknown; do not infer

HSP recommends polling every 15 seconds and reporting “Pending confirmation” after 10 minutes without a safe/finalized/failed result, while retaining identifiers for later recovery. filecite turn0file1

Idempotency strategy

`request_payment(invoice_id)` must:

1. lock the invoice row;
2. verify current state is `DELIVERY_ACCEPTED`;
3. search active attempts;
4. return existing attempt if found;
5. only otherwise call `create_order`;
6. persist returned HSP IDs atomically;
7. commit;
8. return.

If HSP times out after the request may have reached the server, the adapter must prefer recovery by known identifiers or order lookup; it must not blindly create a second commercial obligation.

QA allowlisting gate

Before any integration testing, identify the **actual public outbound IP** used by the backend that will call HSP QA, send it to HSP Operations, and wait for allowlist confirmation. Changing hosting/VPN/proxy can change the IP and require a new allowlist entry. filecite turn0file1

Security model and threat model

Key-custody rule: HashX never receives or stores a raw user private key. Seller invoice signing and buyer HSP signing occur through wallet interfaces.

Policy firewall rule: AI output is a proposal. Deterministic policy code determines whether a transaction is eligible. Financial-signing payloads come from HSP, not from the LLM.

Threat	Attack	Required control
Prompt injection	Seller description tells agent to change recipient	Transaction fields cannot be sourced from free text
Invoice tampering	Payee/amount changed after signature	Recompute EIP-712 digest

Threat	Attack	Required control
Seller impersonation	Attacker signs with own wallet	Expected-counterparty identity binding
Replay	Reuse signed invoice	Seller nonce + invoice state + expiry
Duplicate payment	Concurrent `request_payment` calls	DB lock + unique active attempt
Payload substitution	Friendly UI says $1, wallet signs another amount	Compare displayed summary against actual HSP typed data
Quote expiration	Submit stale signature	Check `quote_expires_at`; reprepare
HSP timeout	Client retries order creation	Persist IDs and query existing payment
Compromised MCP caller	Unauthorized mutation	Authentication, tenant/role checks, rate limits
Server compromise	Attacker rewrites audit record	Append-only audit chain + public commitment
Malicious seller artifact	Crafted parser payload	JSON size/schema limits, no arbitrary code execution
RPC inconsistency	Wrong analytics result	Retry/confirm block range; optionally independent RPC fallback
Secret leakage	Logs contain signatures/tokens	Redaction policy and structured logging

Required security tests include mutated invoice fields, forged signer, expired invoice, reused nonce, wrong seller wallet, malformed JSON, oversized artifact, fake transaction hash, mismatched block range, duplicate payment race, expired HSP quote, timeout recovery, wrong payee, wrong payer, wrong token contract, prompt injection in description and false `payment-safe → finalized` promotion.

Evidence Contract, UX, Demo, and Engineering Plan

Minimal on-chain evidence registry

The contract should **not custody tokens, validate HSP settlement, implement escrow, or attempt to recreate HashKey Merchant**. Its sole purpose is timestamped commitment publication.

Recommended public data:

- `invoiceCommitment`
- issuer address
- `deliveryCommitment` when available
- `receiptCommitment` when settlement is reconciled
- schema/version
- timestamps

Do **not** store invoice plaintext, price, customer identity, report content or private metadata on-chain.

ABI

Function / event	Type	Purpose
`commitInvoice(bytes32)`	function	Register seller's signed invoice digest
`commitDelivery(bytes32,bytes32)`	function	Attach delivery evidence digest
`commitReceipt(bytes32,bytes32)`	function	Attach reconciled receipt digest
`getEvidence(bytes32)`	view	Resolve commitments and issuer
`InvoiceCommitted`	event	Index invoice commitment
`DeliveryCommitted`	event	Index delivery commitment
`ReceiptCommitted`	event	Index receipt commitment

Minimal Solidity interface

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

interface IHashXEvidenceRegistry {
    struct Evidence {
        address issuer;
        bytes32 deliveryCommitment;
        bytes32 receiptCommitment;
        uint64 invoiceCommittedAt;
        uint64 deliveryCommittedAt;
        uint64 receiptCommittedAt;
    }

    event InvoiceCommitted(
        bytes32 indexed invoiceCommitment,
        address indexed issuer
    );

    event DeliveryCommitted(
        bytes32 indexed invoiceCommitment,
        bytes32 indexed deliveryCommitment
    );

    event ReceiptCommitted(
        bytes32 indexed invoiceCommitment,
        bytes32 indexed receiptCommitment
    );
```

```
    function commitInvoice(bytes32 invoiceCommitment) external;

    function commitDelivery(
        bytes32 invoiceCommitment,
        bytes32 deliveryCommitment
    ) external;

    function commitReceipt(
        bytes32 invoiceCommitment,
        bytes32 receiptCommitment
    ) external;

    function getEvidence(bytes32 invoiceCommitment)
        external
        view
        returns (Evidence memory);
}
```

Implementation rules:

- `commitInvoice` rejects zero digests and duplicates.
- only original issuer may append delivery/receipt evidence;
- each evidence slot is write-once;
- no admin can rewrite evidence;
- no token approvals/transfers;
- no upgradeability in V1;
- commitments should include an off-chain random salt when underlying invoice values are guessable.

Gas should remain modest because each commercial object requires only a few fixed-width storage writes. Exact gas consumption must be measured with Foundry against the compiled implementation rather than promised in the PRD. HashKey supports Foundry deployment on testnet through its EVM RPC; mainnet uses chain ID 177. [8]

Dashboard UX

Invoice Workspace

Left column: invoice list with `Draft / Signed / Verified / Delivered / Eligible / Paid / Failed`.

Primary panel: seller, buyer reference, service description, exact service parameters, invoice amount, payee, expiry, signature status.

Actions: create, sign, submit delivery, request verification.

Design principle: a signed invoice is visibly immutable; editing creates a new version.

Trust Inspector

Top: one explicit verdict, `PASS`, `FAIL`, or `PENDING`.

Middle: evidence checks with reasons.

Example:

```
Invoice Signature       PASS
Seller Identity         PASS
Invoice Expiry          PASS
Nonce / Replay          PASS
Artifact Hash           PASS
Requested Address       PASS
Block Range             PASS
Transaction Count       FAIL
```

Bottom: exact evidence digest, verifier version, timestamps, source RPC information and “Why payment is blocked.”

Settlement Explorer

Display:

- HashX payment attempt ID;
- HSP `flow_id`;
- HSP `payment_request_id`;
- merchant name and `attribution_verified`;
- payer;
- payee;
- token contract;
- `token_total_raw`;
- quote expiry;
- HSP state history;
- transaction hash;
- evidence-registry transaction.

Never label `payment-safe` as finalized; HSP explicitly distinguishes those states. filecite turn0file1

Judge-facing demo script

Adversarial test: invoice mutation

Seller signs a `$1` invoice to wallet A. Demo changes `payee` to wallet B.

Expected: `verify_invoice → FAIL / SIGNATURE_INVALID`; `request_payment` inaccessible.

Adversarial test: fake fulfillment

Seller submits report claiming 23 transactions when RPC recomputation finds 17.

Expected: `verify_delivery → FAIL / TX_COUNT_MISMATCH`; payment ineligible.

Adversarial test: duplicate/replay

Call `request_payment` twice or run concurrent requests.

Expected: same existing payment attempt returned; no second HSP order.

Golden path

1. buyer agent requests analytics;
2. seller agent creates and signs invoice;
3. seller delivers JSON;
4. HashX recomputes acceptance evidence;
5. Trust Inspector shows all checks pass;
6. `request_payment` invokes HSP;
7. UI shows actual HSP recipient, payer and raw amount;
8. human approves signing;
9. wallet signs exact HSP EIP-712 data;
10. `finalize_payment`;
11. human approves submission;
12. `submit_payment`;
13. `query_payment`;
14. Settlement Explorer displays actual returned state and tx hash when available;
15. receipt digest committed to HashKey Chain;
16. `get_receipt` returns the complete evidence bundle.

Engineering stack

Layer	Recommendation
Web	Next.js + TypeScript
UI	Tailwind + lightweight component library
API	Node/TypeScript
MCP	Official MCP TypeScript SDK v2
Schema validation	Zod

Layer	Recommendation
DB	PostgreSQL
ORM	Drizzle or Prisma; choose one only
EVM client	viem
Smart contracts	Solidity 0.8.24+
Contract tooling	Foundry
Hashing	Keccak-256 for EVM commitments; SHA-256 for artifact byte hashes
Object storage	Private S3-compatible bucket
Tests	Vitest/Jest + Foundry + Playwright
CI	GitHub Actions
Deployment	frontend + persistent backend with stable outbound IP

The official MCP TypeScript SDK supports Streamable HTTP server deployments; use a persistent backend rather than assuming a purely serverless environment will maintain stable networking/session behavior or HSP allowlisting. [9]

Testing matrix

Layer	Test	Tool	P0
EIP-712	Golden digest recovery	Vitest + viem	Yes
EIP-712	Mutate every signed field	Property/table tests	Yes
Replay	Same nonce twice	Integration	Yes
Invoice DB	Concurrent creation	Postgres integration	Yes
Delivery	Valid analytics	Vitest	Yes
Delivery	Wrong address/range/count	Vitest	Yes
Delivery	Fake tx hashes	RPC integration	Yes
Delivery	Oversized/malformed JSON	Security test	Yes
Payment	Duplicate request race	Postgres integration	Yes
HSP	All 8 tools discovered	Live QA	Yes
HSP	Quote expiration	Live/stub integration	Yes
HSP	Timeout recovery	Adapter integration	Yes
HSP	`payment-safe` handling	Adapter unit	Yes

Layer	Test	Tool	P0
HSP	`payment-finalized` handling	Adapter unit	Yes
Contract	Duplicate invoice rejected	Foundry	Yes
Contract	Nonissuer cannot append	Foundry	Yes
Contract	Evidence write-once	Foundry	Yes
Contract	Fuzz zero/random commitments	Foundry fuzz	Yes
UI	Approver sees exact terms	Playwright	Yes
UI	Failed verification blocks pay	Playwright	Yes
E2E	Golden A2A purchase	Full stack + HSP QA	Critical

CI gates: lint → typecheck → unit tests → Foundry tests → DB integration tests → build. Live HSP tests should be an explicitly invoked protected workflow because they depend on QA access, test assets and external state.

Deployment, Metrics, Risks, Competition, and Sprint Execution

Deployment and validation plan

HashKey's official network documentation currently specifies testnet chain ID `133`, RPC `https://testnet.hsk.xyz`, and mainnet chain ID `177`, RPC `https://mainnet.hsk.xyz`. [10]

HSP QA requires an allowlisted public outbound IP and current live tool schemas. It centrally manages the merchant App Key/App Secret for the hackathon MCP flow, unlike the general HashKey Merchant API model. filecite turn0file1

Gate	Requirement	Status
G0	Stable backend egress IP	**Live validation required**
G1	HSP Operations allowlist confirmation	**Live validation required**
G2	MCP endpoint reachable	**Live validation required**
G3	All 8 HSP tools exposed	**Live validation required**
G4	Live `get_chain_config` stored	**Live validation required**
G5	Test wallet funded with exact token	**Manual/live validation required**
G6	Required Permit2/token approval completed if applicable	**Manual/live validation required**
G7	Successful `$1`-class QA payment	**Critical live validation**
G8	Status recovery demonstrated	**Critical live validation**

Gate	Requirement	Status
G9	Evidence registry on testnet	Required engineering gate
G10	Mainnet deployment interpretation confirmed	**Organizer confirmation required**
G11	Minimal mainnet deployment if required	**Depends on G10**
G12	Demo recording + README + links	Required submission gate

The HSP guide says prerequisite approvals and funding must be manually completed by the team and that the agent should not perform those preparation transactions on the team's behalf. filecite turn0file1

Timeline to November 1, 21:30 IST

Window	Owner	Critical deliverable
Oct 30 evening	Integration lead	Stable egress IP + HSP allowlist request
Oct 30 evening	Protocol lead	Freeze EIP-712 and DB migration
Oct 30 night	Smart-contract lead	Evidence Registry + Foundry tests
Oct 30 night	Backend lead	Invoice create/sign/verify APIs
Oct 31 morning	Verification lead	Analytics verifier + RPC recomputation
Oct 31 morning	Integration lead	Inspect all live HSP schemas
Oct 31 afternoon	Backend lead	HSP adapter + idempotent payment state machine
Oct 31 afternoon	Frontend lead	Invoice Workspace + Trust Inspector
Oct 31 evening	Team	First full golden path in staging
Oct 31 night	Security/test lead	Adversarial suite + duplicate race test
Nov 1 morning	Integration lead	Real HSP QA payment and recovery proof
Nov 1 morning	Contract lead	Testnet deployment; mainnet decision
Nov 1 afternoon	Frontend + pitch	Settlement Explorer + demo polish
Nov 1 16:00–18:00	Team	Recorded clean run; screenshots/tx links
Nov 1 18:00–20:00	Documentation lead	README, architecture, limitations, build submission
Nov 1 before 21:30	Team lead	DoraHacks final submission verification

Do not postpone the first HSP payment until the end; it is the highest external dependency.

Organizer submission checklist

Use the user's supplied hackathon rules as the operational deadline and requirements baseline, while obtaining direct organizer confirmation for the conflict concerning mainnet deployment. The generic DoraHacks platform indicates that hackathon organizers control eligibility and judging rules, so organizer clarification is authoritative for ambiguous requirements. [11]

Submission package:

- public GitHub repository;
- clear README;
- problem / solution;
- architecture diagram;
- MCP tool documentation;
- live/deployed frontend;
- HashKey Chain deployment address;
- HSP QA evidence;
- real payment identifiers and tx hash where available;
- test results;
- demo video;
- three adversarial scenarios;
- golden path;
- explicit limitations;
- team members;
- track selection;
- setup instructions reproducible by judges;
- organizer-confirmed mainnet status.

Success metrics

Engineering acceptance

Metric	Target
Signed-field mutation tests rejected	100%
Known invalid delivery tests rejected	100%
Valid deterministic report accepted	100%
Duplicate-payment race creates >1 active HSP order	0
Uncertain HSP result produces blind re-order	0
`payment-safe` incorrectly called final	0
Raw private keys stored by HashX	0
Golden-path HSP QA payment	≥1 real completed run

Metric	Target
Evidence contract Foundry test pass rate	100%
Demo workflow	<5 minutes with no hidden manual database edits

Judge-facing acceptance

A judge must be able to answer all five questions from the demo:

1. What was purchased?
2. Who signed the request?
3. What objective evidence showed delivery?
4. What exact payment did the human authorize?
5. What did HSP/HashKey Chain actually settle?

If any answer relies on “trust our dashboard,” the project has failed its own thesis.

Priority risks and contingencies

Risk	Severity	Mitigation	Contingency
HSP allowlisting delayed	Critical	Submit IP immediately	Record all non-HSP flows; keep adapter ready, but clearly disclose no live payment
HSP live schemas differ	High	Dynamic inspection	Thin translation adapter; never hard-code guessed fields
Mainnet rule ambiguous	Critical	Ask organizer now	Deploy harmless Evidence Registry to 177 if permitted and affordable
Payment QA token unavailable	High	Prepare wallet early	Use organizer-recommended faucet/support
Delivery verifier too complex	Medium	Narrow metrics to count + sample proof	Drop advanced aggregates, keep deterministic minimum
Invoice identity claim overreaches	High	Call it wallet control/trusted counterparties	Explicitly avoid “KYC verified”
Competitor overlap	High	Lead with fulfillment verification + commercial evidence	Emphasize negative tests and end-to-end binding
UI consumes time	Medium	Three screens only	Use functional admin-style UI
Contract bug	Medium	Commitment-only design	Keep contract out of funds path

Risk	Severity	Mitigation	Contingency
Agent integration instability	Medium	MCP is optional front door to same API	Demo with one known MCP client
Seller nonpayment exposure	Product risk	Clearly disclose delivery-first model	Future escrow roadmap, not MVP
RPC mismatch/rate limits	Medium	Cache + bounded ranges	Use small demo block ranges

Competitor comparison

The comparison below is based on public project descriptions and repositories; it should not be treated as a claim about unreleased Mumbai judging results.

Project	Core proposition	Strength	Gap HashX should exploit
Sentinel	Policy-gated/controlled AI financial execution	Strong security/risk story	HashX focuses on verifying the commercial obligation and fulfillment, not merely execution permission
Sumplus	On-chain mandates, compliance attestations, HSP receipts	Strong bounded-spend/compliance narrative	HashX verifies seller invoice integrity and objective service delivery before authorization
PayPort	Drop-in HashKey stablecoin checkout SDK	Excellent merchant integration simplicity	HashX is not checkout abstraction; it adds pre-payment commercial verification
AllScale	Merchant-side agent screening and settlement verification	Previous winning agent-commerce concept	HashX moves trust to the **transaction obligation**: signed terms + fulfillment evidence + buyer-side decision
Astrail	AI travel workflow with human-approved HSP payment	Strong end-user story	HashX is reusable infrastructure and makes service verification first-class

Sumplus publicly describes an agent whose payments are governed by an on-chain mandate and HSP receipts, including compliance attestations; this means HashX should not pitch “controlled agent spending” as novel. [12]

PayPort publicly describes a drop-in SDK with a full backend, dashboard, HashKey QA integration and webhook-based settlement, so “easy HashKey payments for merchants” is already occupied territory. [13]

AllScale's repository describes a canonical checkout, EIP-712 gateway signature, ERC-20 settlement and receipt verification, while explicitly disclosing that KYT/KYC integrations in the hackathon build were mock/demo gates. Its prior first-place result makes it the most strategically important precedent for HashX. [14]

Astrail already demonstrated a vertical AI workflow with real HashKey HSP/x402 testnet payment proof while its hotel fulfillment remained mock-only; **HashX should directly turn that missing fulfillment-verification problem into its differentiator.** [15]

The directly supplied Sentinel DoraHacks URL should remain in the competitor appendix even though DoraHacks did not expose the page to automated retrieval during this research: Sentinel build 50437. Claims about Sentinel should therefore be treated as requiring manual confirmation from the live build page before final pitch language.

Recommended team roles and immediate sprint

For a four-person team, roles should overlap deliberately:

Role	Owner profile	Immediate responsibility
Protocol / Backend Lead	strongest TypeScript + system design developer	invoice schema, DB state machine, MCP
HSP / Integration Lead	strongest debugging/integration developer	allowlisting, wallet, HSP live schemas, real payment
Smart Contract / Security Lead	strongest Solidity/EVM developer	Evidence Registry, EIP-712 test vectors, adversarial tests
Frontend / Product / Demo Lead	strongest UI + communication developer	dashboard, UX, README, video, pitch

For three people, merge contract/security into backend. For two, one owns backend/HSP/contracts and one owns frontend/testing/pitch. **Do not assign a dedicated "AI engineer" unless all payment-critical engineering is covered; V1 does not require model training.**

Immediate sprint ownership:

Integration lead — now: secure stable egress IP, send allowlist request, connect to HSP, record tool schemas, obtain correct test token/approvals, attempt first `$1` payment before anything cosmetic. The HSP guide explicitly makes allowlisting and manual wallet preparation prerequisites. filecite turn0file1

Backend lead — now: create repository structure, Postgres migrations, canonical invoice serializer, EIP-712 implementation, invoice verification, state transition library and idempotency layer.

Security/contract lead — now: build commitment-only registry, deploy testnet version, Foundry unit/fuzz tests, EIP-712 mutation test vectors and concurrent duplicate-payment test harness.

Frontend/product lead — now: build only the three dashboard screens, with actual backend data; prepare attack-demo buttons that invoke genuine mutation/failure scenarios rather than mocked animation.

The project should use HashKey's current official developer documentation for network configuration, the supplied October 30 HSP Agentic Payment guide as the operational HSP authority, current HashKey

Merchant documentation for broader merchant context, and public DoraHacks/GitHub projects for competitor benchmarking. HashKey Merchant's general product already exposes one-time/reusable orders, multi-chain/multi-token support, HMAC-authenticated merchant APIs and signed webhooks, reinforcing the decision to make **verification**, not payment plumbing, HashX's core intellectual contribution. [5]

Primary source links: HashKey Chain documentation, HashKey Chain network information, HashKey Merchant documentation, EIP-712, MCP TypeScript SDK, Sentinel DoraHacks build. The uploaded October 30 HSP guide should be treated as the primary hackathon-specific payment execution reference. 🀫filecite🀫turn0file1🀫

Final product requirement: HashX Global V1 is successful only when it demonstrates that a manipulated commercial request is rejected, an objectively invalid delivery is rejected, duplicate payment attempts do not create duplicate obligations, and a valid commercial exchange can proceed through a real user-approved HSP payment whose resulting state is reconciled accurately. Everything else is secondary.

---

[1] [7] [10] HashKey Docs
https://docs.hskchain.net/docs/Build-on-HashKey-Chain/network-info?utm_source=chatgpt.com

[2] HSK Chain News – Telegram
https://t.me/s/hashkeyhsk?utm_source=chatgpt.com

[3] EIP-712: Typed structured data hashing and signing
https://eips.ethereum.org/EIPS/eip-712?utm_source=chatgpt.com

[4] [9] GitHub - modelcontextprotocol/typescript-sdk: The official TypeScript SDK for Model Context Protocol servers and clients · GitHub
https://github.com/modelcontextprotocol/typescript-sdk?utm_source=chatgpt.com

[5] HashKey Merchant — API Documentation
https://hashkey-merchant.github.io/index.html?utm_source=chatgpt.com

[6] ERC-1271: Standard Signature Validation Method for Contracts
https://eips.ethereum.org/EIPS/eip-1271?utm_source=chatgpt.com

[8] HashKey Docs
https://docs.hskchain.net/docs/Developer-QuickStart?utm_source=chatgpt.com

[11] Organizer Dashboard | DoraHacks
https://dashboard.dorahacks.io/hackathon/%24demo/admin?utm_source=chatgpt.com

[12] GitHub - sumplus-real/sumplus-hsk-mandate · GitHub
https://github.com/sumplus-real/sumplus-hsk-mandate?utm_source=chatgpt.com

[13] GitHub - ArthFi/PayPort: PayPort is a drop-in payment SDK for HashKey Chain. Add it to any website with a single